Paper 1 (SAR-CRP v2 Core) — Full Q1 Experiment Suite Report

August 8, 2026

1 Setup

Hardware. All computation ran on the remote CPU server (`a100-B`, `story_research` conda env); the laptop was used only to edit code (spec §51's disclosure requirement). CPU: Intel(R) Xeon(R) Gold 6226R @ 2.90GHz, 2 sockets × 16 cores × 2 threads/core = 64 logical CPUs, 2 NUMA nodes. Software: Python 3.11.15, PyTorch 2.13.0+cpu (CPU-only build — CRP_RL inference in this report ran on CPU, not GPU), SciPy 1.17.1, NumPy 2.4.6.

Seed policy. `sarcrp.seed_policy` distinguishes `DEV_SEEDS` (0–9, inspected by hand while diagnosing earlier bugs — e.g. the MVP report's seed-7 trace) from `REPORT_SEEDS` (20–39, fresh, never inspected during development). Every report-grade number below uses `REPORT_SEEDS` (spec §23.6's ≥20-seed minimum), never `DEV_SEEDS`.

2 Baselines and Ablations Implemented

ID	Description
B1 Static Plan	Never replan (spec §22)
B2 Full Reoptimization	Re-solve the whole remaining problem on every event; pluggable solver (greedy or real CRP_RL)
B3 Periodic Replanning	Re-solve every k-th event, otherwise keep the current plan
B4 Event-triggered, No Stability	Same trigger + freeze horizon as SAR-CRP, but $\lambda = \mu = 0$ (objective is operational cost only)
B5 MPC Receding Horizon	Freeze a fixed-size prefix, unconditionally re-solve the tail against the post-frozen-prefix state every event
B6 SAR-CRP Core	Proposed method: trigger + freeze horizon + local-search repair + stability-aware objective
A1 No Trigger	Always replan (removes the impact threshold)
A2 No Freeze Horizon	Allow the repair to touch the entire plan
A3 No Stability Cost	Optimize operational cost only ($\lambda = 0$)
A4 No Local Search	Minimal feasibility repair or full reoptimization only
A5 No Data Confidence Penalty	Ignore state confidence in the objective
A6 No Blocking Impact	Impact estimate excludes $I_{blocking}$

Table 1: B1–B6 (spec §22, §40) and A1–A6 (spec §25), as implemented in `sarcrp.baselines` and `sarcrp.ablations`.

3 Experiment 1: Main Factorial Results

Factorial design (spec §23): uncertainty level × freeze size × λ = 3 × 3 × 3, all 6 methods (B1–B6), 20 seeds (`REPORT_SEEDS`) = 3240 episodes total, on `small_layout_mvp.json`. Run via `run_experiment1.py` (commit `2617a50`, duration 481.1s, logged in `experiments/logs/run_log.jsonl`).

Correctness note. Two real bugs were caught and fixed against this experiment's own output before treating any number below as final: (1) the significance-test helper collapsed the full 27-cell factorial grid into one seed-keyed dictionary without filtering first, so a duplicate seed silently kept whichever grid cell was iterated last (`freeze_size=5`, $\lambda = 1.0$) instead of any deliberate operating point; and (2) the MPC baseline (B5) solved its tail against the *original, untouched* yard state, so its own frozen prefix's moves were silently re-planned again by the tail, making `is_plan_valid` reject nearly every episode and the $M_{\text{inf}} = 10^6$ invalidity penalty dominate its reported cost. Both are fixed (Wilcoxon now filters to one explicit `(uncertainty_level, freeze_size=3, λ=1.0)` cell — spec §48's own SAR-CRP defaults — per uncertainty level; MPC now shadow-applies its frozen prefix before solving the tail). This experiment was re-run after the MPC fix; the numbers below are from that re-run.

Uncertainty	Method	Total cost (mean)	Op. cost (mean)	Changed actions (mean)	Fallback rate
low	Static	7.042	7.042	0.00	1.000
low	Full Reoptimization	19.045	6.700	59.10	0.000
low	Periodic	11.118	6.991	18.05	0.851
low	Event-triggered, No Stability	7.042	7.042	0.00	1.000
low	MPC Receding Horizon	16.632	7.547	49.45	0.000
low	SAR-CRP	7.042	7.042	0.00	1.000
medium	Static	7.052	7.052	0.00	1.000
medium	Full Reoptimization	21.866	6.612	71.50	0.000
medium	Periodic	11.644	6.959	21.35	0.850
medium	Event-triggered, No Stability	7.052	7.052	0.00	1.000
medium	MPC Receding Horizon	17.789	7.560	54.10	0.000
medium	SAR-CRP	7.052	7.052	0.00	1.000
high	Static	7.059	7.059	0.00	1.000
high	Full Reoptimization	23.333	6.523	79.45	0.000
high	Periodic	11.612	6.781	21.35	0.850
high	Event-triggered, No Stability	7.059	7.059	0.00	1.000
high	MPC Receding Horizon	18.888	7.747	59.00	0.000
high	SAR-CRP	7.059	7.059	0.00	1.000

Table 2: Experiment 1: mean over freeze_size × λ × 20 seeds (n = 180 per row). Source: `experiments/results/experiment1_results.csv`, post MPC fix (commit 2617a50).

SAR-CRP vs. each baseline (spec §23.6 protocol: paired Wilcoxon signed-rank, `zero_method="pratt"`, Holm–Bonferroni correction across the 5 comparisons, Cliff's delta), at the default operating point (`freeze_size=3`, $\lambda = 1.0$), per uncertainty level:

SAR-CRP ties Static and B4 exactly (0/20 nonzero pairs) at every uncertainty level. This is the same real, explainable result the MVP report already documented, not a new bug: spec §11.2's *RetrievalDelay*(P) term only sums position over *urgent* containers, and only `URGENT_INSERTION` events populate the urgent set; a pure `ORDER_SWAP`/ETA/`PROBABILITY_UPDATE` event changes the queue but changes no plan's operational cost, so there is nothing for repair to improve on. §20 SC4 (below) confirms this instance's mean event impact (0.090) sits well under $\theta_{\text{impact}} = 0.30$, so the trigger essentially never fires on this benchmark regardless of `freeze_size`/λ

Uncertainty	Baseline	p-value	n_{nonzero}/n	Holm-sig.	Cliff's δ
low	Static	1.0000	0/20	No	0.000
low	Full Reoptimization	0.0000	20/20	Yes	−1.000
low	Periodic	0.0000	20/20	Yes	−1.000
low	Event-triggered, No Stability	1.0000	0/20	No	0.000
low	MPC Receding Horizon	0.0000	20/20	Yes	−1.000
medium	Static	1.0000	0/20	No	0.000
medium	Full Reoptimization	0.0000	20/20	Yes	−1.000
medium	Periodic	0.0000	20/20	Yes	−1.000
medium	Event-triggered, No Stability	1.0000	0/20	No	0.000
medium	MPC Receding Horizon	0.0000	20/20	Yes	−1.000
high	Static	1.0000	0/20	No	0.000
high	Full Reoptimization	0.0000	20/20	Yes	−1.000
high	Periodic	0.0000	20/20	Yes	−1.000
high	Event-triggered, No Stability	1.0000	0/20	No	0.000
high	MPC Receding Horizon	0.0000	20/20	Yes	−1.000

Table 3: Source: `experiments/results/experiment1_significance.csv`, post MPC fix (commit 2617a50).

– consistent with Task 27/28's own finding that varying those two factors produced no observable behavioral difference here. SAR-CRP beats Full Reoptimization, Periodic, and MPC with $p < 0.0001$ (Holm-significant) and Cliff's $\delta = -1.0$ at every uncertainty level: those three baselines all incur real reoptimization/instability cost every event, while SAR-CRP correctly recognizes (via the trigger) that repairing is not worth it on this benchmark and defaults to Static's behavior instead.

4 Experiment 3: Cross-Layout

Layout A (`small_layout_mvp.json`, 10 containers, timeout 1.0s), Layout B (`layout_b.json`, 15 containers, timeout 5.0s), Layout C (`layout_c.json`, 24 containers, timeout 30.0s) — spec §17's own size-based timeout tiers, same hyperparameters (spec §48 defaults) otherwise, no per-layout tuning. 20 seeds (`REPORT_SEEDS`), 3 methods (Static, Full Reoptimization, SAR-CRP).

Method	Performance drop, Layout B vs. A	Performance drop, Layout C vs. A
Static	+141.5%	+325.6%
Full Reoptimization	+43.6%	+103.2%
SAR-CRP	+141.5%	+325.6%

Table 4: Relative total-cost change vs. Layout A (spec §24.5/§50). Source: `experiments/results/cross_layout_results.csv`.

SAR-CRP's performance drop tracks Static's exactly on both larger layouts, for the same reason as Experiment 1: it never crosses the impact trigger threshold on any of the three layouts at these defaults, so it always falls back to Static's plan. Full Reoptimization's smaller relative drop reflects that its absolute total cost already starts much higher on Layout A (it pays a large stability cost every event regardless of layout size), so the same absolute cost increase on larger layouts is a proportionally smaller jump.

5 Experiment 4: Data Confidence Sensitivity

Confidence pinned per event (spec §23's own protocol for isolating confidence's effect from severity) at 1.0/0.7/0.4/0.2, 20 seeds (`REPORT_SEEDS`), on `small_layout_mvp.json`.

Method	Total cost mean, by fixed confidence			
	1.0	0.7	0.4	0.2
Static	7.049	7.049	7.049	7.049
Full Reoptimization	20.393	21.493	22.593	23.326
SAR-CRP	7.049	7.049	7.049	7.049

Table 5: Source: `experiments/results/experiment4_results.csv`.

Full Reoptimization's total cost rises monotonically as confidence drops (20.39 → 23.33, spec §11.4's data-confidence penalty scaling with how much the relied-upon plan information has changed) — the mechanism spec §23 Experiment 4 is meant to exercise works as intended. Static and SAR-CRP show no confidence sensitivity on this instance for the same reason as Experiments 1 and 3: both never change their plan (`changed_actions_total`=0 in every row), so `data_confidence_cost` — which is computed from the delta between the new and old plan — is trivially zero regardless of the confidence level fed in.

6 Sanity Checks (spec §20, §49)

- SC1 (not too easy): **PASS** — the base plan requires real relocations (bottom-first round-robin retrieval order).
- SC2 (not too hard): **PASS** — measured on Full Reoptimization's incomplete-plan rate, not Static's trivial fallback rate.
- SC3 event-type frequency: `URGENT_INSERTION`=0.233, `ORDER_SWAP`=0.400, `PROBABILITY_UPDATE`=0.144, `ETA_LATE`=0.111, `ETA_EARLY`=0.089, `STALE_INFORMATION`=0.022.
- SC4 (mean impact in $[0.2, 0.8]$): **FAIL** (mean=0.090). This is the direct explanation for Experiments 1/3/4's SAR-CRP-ties-Static finding above: this benchmark instance's synthetic event stream produces impact scores well below $\theta_{\text{impact}} = 0.30$ on average, so the trigger rarely fires. Flagged here rather than hidden — see Limitations.

7 Ground Truth

Small, exhaustive (spec §21.1). On `tiny_ground_truth.json` (3 stacks × 2 containers, 6 total, branch-and-bound exhaustive search, no randomness so no seed sweep needed): optimal = 4 relocations, greedy = 5 relocations, optimality gap = 25.0%.

Offline proxy, larger layouts (spec §21.2). Full Reoptimization run with a 300s extended timeout as an offline high-quality *proxy* for the optimum (never called the true optimum, per spec §21.2's explicit wording) on Layouts B and C: gap vs. proxy = 0.0% on both (17 vs. 17 relocations on Layout B, 30 vs. 30 on Layout C) — expected, since the greedy solver is non-iterative and does not improve with more wall-clock time.

Method	Invalid rate	Timeout rate	Runtime P95 (s)	Stability cost (mean)
Static	0.0000	0.0000	0.00000	0.0000
Full Reoptimization	0.0000	0.0000	0.00009	14.1735
SAR-CRP	0.0000	0.0000	0.26330	0.0000
MPC (post-fix)	0.0000	0.0000	0.00015	9.3986

Table 6: `small_layout_mvp.json`, `REPORT_SEEDS` (20 seeds).

8 Metrics Completeness (spec §24, Task 30)

`invalid_rate` was 0.0 for every method *after* the MPC fix above — before that fix, MPC alone showed `invalid_rate`=1.0 on a 3-seed spot check, which is exactly how the bug was caught: this is the first time `is_plan_valid`/M_{inf} has actually been exercised as a live check in this codebase rather than dead code (every call site had passed `is_valid=True` unconditionally through Task 6–29). SAR-CRP's own `runtime_p95_sec` (0.263s) is noticeably higher than Full Reoptimization's (0.00009s) despite SAR-CRP mostly falling back to KEEP on this instance — its impact estimation and trigger check still run on every event even when the outer decision is KEEP.

9 Real Solver Comparison (Task 33/34, spec §43)

`sarcrp.crp_rl_adapter` wraps the real trained CRP_RL model (Shin et al. 2026, MIT-licensed code) behind the `solve_crp` interface, reusing only its relocation *decisions* – the resulting plan is scored by this project's own `objective.py`, not CRP_RL's own travel-time cost model.

Out-of-distribution sanity check only (not evidence of relative quality). `small_layout_mvp.json` has 10 containers, far below CRP_RL's training distribution (35–70 containers per its own `baselines/test.py`). Greedy total_cost_mean over seeds 0–4: [15.62, 19.45, 20.45, 17.13, 25.51]; CRP_RL: [28.95, 38.65, 44.54, 29.11, 34.81]. Greedy is cheaper here, but the comparison is out-of-distribution and must not be read as evidence about CRP_RL's true quality.

In-distribution comparison (the one comparison usable as evidence). A dedicated 50-container instance (`crp_rl_scale_instance.json`, 10 stacks × 5 containers, bottom-first round-robin, inside CRP_RL's [35, 70]-container trained range) was generated for this purpose. Greedy total_cost_mean over seeds 0–4: [78.30, 84.16, 63.29, 75.86, 75.40] (mean ≈75.4); CRP_RL: [155.17, 151.17, 177.74, 142.64, 155.84] (mean ≈156.5). Greedy still outperforms the real trained model by a wide margin (≈48% lower mean total cost) even within CRP_RL's own trained size range. **Caveat:** because the adapter only reuses CRP_RL's relocation decisions and rescoring happens under SAR-CRP's own objective (not the metric CRP_RL was trained to minimize), this is evidence that CRP_RL's relocation choices do not beat the greedy heuristic *under this objective*, not a general claim that CRP_RL is a worse CRP solver in an absolute sense.

10 Reproducibility

Every run cited above is recorded in `experiments/logs/run_log.jsonl` (git commit, hostname, exact params, wall-clock duration, output paths) – gitignored locally, one JSON line per invocation. Key entries: `run_cross_layout.py` (13.26s), `run_experiment4.py` (41.79s), `run_experiment1.py` (481.06s, re-run after the MPC fix, git commit `2617a50`).

11 Limitations

- **Experiment 5** (operator-acceptance study) is intentionally omitted, per spec §23's own framing of it as optional – noted here explicitly rather than silently dropped.
- **This benchmark instance rarely triggers SAR-CRP's repair path.** SC4 (mean impact 0.090 vs. $\theta_{\text{impact}} = 0.30$) is the root cause behind Experiments 1/3/4 all showing SAR-CRP tied to Static: the synthetic event generator on this instance's parameters essentially never produces an event severe enough to cross the trigger. The comparisons against Full Reoptimization/Periodic/MPC are still informative (they show SAR-CRP correctly avoids unnecessary churn), but this suite does not yet demonstrate a scenario where SAR-CRP's repair mechanism produces a strict, active win over Static. A benchmark instance/event-parameter combination that reliably crosses the trigger is left to future work.
- **CRP_RL comparison caveat** as stated in §9 above: rescoring under SAR-CRP's own objective, not CRP_RL's native metric.
- **Base-paper citations** (Shin et al. 2026, Zhou & Zhang 2024, Zhang et al. 2025) were verified for DOI/venue in an earlier phase of this project but are not re-verified in this report.

12 Conclusion

The full baseline suite (B1–B6), all six ablations (A1–A6), the statistical protocol (spec §23.6), cross-layout generalization, ground truth/offline-proxy comparisons, and a real trained-model comparison are now all implemented and exercised with real, seeded runs rather than placeholders. Two genuine implementation bugs were caught and fixed directly from this experiment suite's own output before being reported (the significance-test grid-cell aggregation bug, and the MPC tail-state bug) – both are documented above rather than silently corrected. The main open finding is that this specific benchmark instance's default event parameters keep SAR-CRP's trigger from firing in practice; the suite is otherwise ready to be pointed at a higher-impact benchmark instance as the natural next step.